UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

(Universidad del Perú, Decana de América)

FACULTAD DE INGENIERÍA DE SISTEMAS E INFORMÁTICA

DISEÑO DE SISTEMAS DE INFORMACIÓN

Caso Asignado	B - Sistema de Mesa de Ayuda de TI (Service Desk)
Organización	Oficina de Tecnologías de la Información de una entidad pública
Integrantes del equipo	Granda Salazar, Rodrigo Christopher (24200017) Quezada Pairazaman, Sebastian Emanuel (24200030) Quispe Arango, Juan Pablo (24200121) Saldaña Sánchez, Luis Mario (24200038) Treviño Alvarado, Milagros Sofia (24200219)
Docente	Andrade Mogollon, Teodoro
Semestre académico	2026-2
Versión del documento	v1.2 - 07/10/26

Índice de contenido

1. Resumen Ejecutivo
2. Introducción
- 2.1 Contexto organizacional
- 2.2 Problema y justificación
- 2.3 Objetivos
- 2.4 Alcance, supuestos y restricciones
  - 2.4.1 Alcance Incluido (In-Scope - Vertical Slice)
  - 2.4.2 Alcance Excluido (Out-of-Scope)
  - 2.4.3 Restricciones Técnicas y Metodológicas
  - 2.4.4 Supuestos Operativos

3. Análisis (Fase de Inicio)
- 3.1 Requisitos funcionales y no funcionales
- 3.2 Modelo de procesos de negocio (BPMN 2.0)
- 3.3 Modelo de Casos de Uso
- 3.4 Modelo de Dominio

4. Diseño (Fase de Elaboración)
- 4.1 Arquitectura de software - Vista 4+1
  - 4.1.1 Vista Lógica (Logical View)
  - 4.1.2 Vista de Procesos (Process View)
  - 4.1.3 Vista de Desarrollo (Development View)
  - 4.1.4 Vista Física / Despliegue (Physical View)
  - 4.1.5 Escenarios (+1)
- 4.2 Diagrama de clases de diseño
- 4.3 Diagrama de secuencia
  - 4.3.1 Catálogo de Líneas de Vida Participantes
  - 4.3.2 Justificación de Fragmentos Combinados (UML 2.5.1)
- 4.4 Diagrama de estados
  - 4.4.1. Descripción de los estados y sus actividades internas
  - 4.4.2. Matriz detallada de transiciones, disparadores y guardas
- 4.5 Diagrama de Componentes
  - 4.5.1 Catálogo de Componentes e Interfaces del Sistema
  - 4.5.2 Mecanismo de Acoplamiento y Flujo de Interconexión
  - 4.5.3 Topología de Despliegue Físico Integrada

Sistema de Mesa de Ayuda de TI (Service Desk)

1. Resumen Ejecutivo

—

2. Introducción

2.1 Contexto organizacional

El presente proyecto se desarrolla en el marco de la **Oficina General de Tecnologías de la Información (OGTI)** de una entidad pública del sector gubernamental peruano (Ministerio / Organismo Público Descentralizado). La OGTI tiene a su cargo la gobernanza digital, la administración de la infraestructura tecnológica, las redes de comunicaciones, las bases de datos y la provisión de soporte técnico a más de 1,200 usuarios internos (servidores públicos) distribuidos en sedes centrales y desconcentradas. Operativamente, la OGTI estructura la atención de incidencias en tres niveles técnicos:

- **Unidad de Mesa de Ayuda (Soporte Nivel 1 - N1):** Punto único de contacto (SPOC) encargado del triaje, recepción, registro inicial, diagnóstico preliminar y resolución de incidencias de baja complejidad (restablecimiento de contraseñas de correo institucional, soporte ofimático y configuración inicial de periféricos).
- **Unidad de Infraestructura y Soporte de Campo (Soporte Nivel 2 - N2):** Equipo técnico especializado en mantenimiento correctivo y configuración *in situ* de hardware, sistemas operativos, servidores de impresión, conectividad cableada/WiFi y software de gestión documental.
- **Unidad de Arquitectura de Sistemas y Seguridad Digital (Soporte Especializado Nivel 3 - N3):** Especialistas en bases de datos (DBA), ingenieros de redes/seguridad perimetral, desarrolladores de software y enlace con proveedores externos para incidencias de alta criticidad, bugs del core del negocio y caídas de infraestructura central.

2.2 Problema y justificación

Actualmente, la OGTI opera bajo un modelo de atención artesanal sustentado en hojas de cálculo compartidas, correos electrónicos y mensajería instantánea. Dicho modelo genera severas deficiencias operativas analizadas bajo el marco de **ITIL v4 (Práctica de Gestión de Incidentes y Gestión de Niveles de Servicio)**:

1. **Ausencia de Cálculo Formal y Monitoreo de SLAs:** No existe un mecanismo sistematizado que determine y controle la fecha/hora límite contractual de atención según la criticidad del incidente. Se pierde visibilidad del vencimiento de plazos.
2. **Escalamiento Discrecional y Cuellos de Botella:** El paso de un incidente de Nivel 1 a Nivel 2 o Nivel 3 se ejecuta manualmente a criterio personal del analista de turno,

generando acumulación de tickets sin resolver en N1 o sobrecarga innecesaria en N3 por mala categorización.

3. **Pérdida de Trazabilidad e Incumplimiento Normativo:** Los incidentes críticos (ej. fallo en el Sistema Integrado de Gestión Administrativa o caída del enlace principal) exceden los tiempos máximos permitidos por los Acuerdos de Nivel de Servicio (SLA) sin que se emita ninguna alerta temprana a la jefatura de la OGTI.
4. **Falta de Desacoplamiento entre Tipos de Incidencia y Reglas de Negocio:** Cada categoría de incidente (Hardware, Software, Redes, Accesos) requiere parámetros de SLA, prioridades base y protocolos de atención diferenciados; sin embargo, hoy se procesan con una plantilla genérica sin polimorfismo.

Matriz de Criticidad ITIL v4: Impacto, Urgencia y Prioridad

En concordancia con ITIL v4, la prioridad de una incidencia resulta de la intersección entre el **Impacto** (magnitud del perjuicio sobre la continuidad de los servicios o volumen de usuarios afectados) y la **Urgencia** (rapidez requerida para resolver el incidente antes de causar daño mayor).

Impacto \ Urgencia	**Alta (A)**	**Media (M)**	**Baja (B)**
Alto (A) (Afecta servicios críticos o > 50 usuarios)	**P1 - Crítica**	**P2 - Alta**	**P3 - Media**
Medio (M) (Afecta una oficina completa o 10 a 50 usuarios)	**P2 - Alta**	**P3 - Media**	**P4 - Baja**
Bajo (B) (Afecta a un solo usuario o función no esencial)	**P3 - Media**	**P4 - Baja**	**P4 - Baja**

Matriz de Acuerdos de Nivel de Servicio (SLA) y Política de Escalamiento Jerárquico

Los SLAs formalizan los tiempos máximos tolerables para el **Tiempo de Respuesta (SLA-R)** y el **Tiempo de Resolución (SLA-T)**.

Prioridad	**Nivel Inicial Asignado**	**SLA Tiempo de Respuesta (SLA-R)**	**SLA Tiempo de Resolución (SLA-T)**	**Umbral de Alerta Temprana (Observer)**	**Regla de Escalamiento Automático (Chain of Responsibility)**

P1 (Crítica)	Nivel 1 / Nivel 2	≤ 15 min	≤ 2 h	Al consumir el 75% del SLA-T (1.5 h)	Si expira SLA-T o no hay resolución en N1/N2 → Escalamiento forzoso a **Nivel 3**
P2 (Alta)	Nivel 1	≤ 30 min	≤ 4 h	Al consumir el 75% del SLA-T (3.0 h)	Si expira SLA-T en N1 → Escalamiento a **Nivel 2**; si expira en N2 → Escalamiento a **Nivel 3**
P3 (Media)	Nivel 1	≤ 1 h	≤ 8 h	Al consumir el 80% del SLA-T (6.4 h)	Si expira SLA-T en N1 → Escalamiento a **Nivel 2**
P4 (Baja)	Nivel 1	≤ 2 h	≤ 24 h	Al consumir el 85% del SLA-T (20.4 h)	Si expira SLA-T en N1 → Escalamiento a **Nivel 2**

2.3 Objetivos

Objetivo General

Diseñar e implementar un prototipo vertical de un Sistema de Mesa de Ayuda de TI (Service Desk) bajo las buenas prácticas de ITIL v4, que automatice la recepción, cálculo y monitoreo de SLAs, así como el escalamiento jerárquico de incidentes mediante patrones de diseño de software.

Objetivos Específicos

1. **Analizar** los procesos operativos de soporte de la OGTI y formalizar los requisitos funcionales y no funcionales bajo el estándar internacional ISO/IEC 25010:2011.
2. **Diseñar** la arquitectura lógica del sistema mediante el modelo de vistas 4+1 (Kruchten) y artefactos UML 2.5.1, aplicando los patrones GoF *Factory Method*, *Chain of Responsibility* y *Observer*.

3. **Implementar** un prototipo vertical (*vertical slice*) en Node.js/TypeScript que cubra de punta a punta el caso de uso crítico de registro y escalamiento automatizado de incidencias por vencimiento de SLA.
4. **Probar** la solución mediante pruebas unitarias de caja blanca (cobertura de ramas y sentencias) y pruebas funcionales de caja negra (particiones de equivalencia y análisis de valores límite temporales).

2.4 Alcance, supuestos y restricciones

2.4.1 Alcance Incluido (In-Scope - Vertical Slice)

El proyecto se enfoca en un prototipo vertical de alta fidelidad que garantice trazabilidad punta a punta:

- **Flujo E2E Crítico:** Implementación integral del CU *"Registrar incidencia y escalarla automáticamente al vencer SLA"*.
- **Pila Arquitectónica Completa:**
    - *Capa de Presentación / API REST:* Endpoints para creación de incidencias, verificación de vencimiento y consulta de estado.
    - *Capa de Aplicación / Servicios:* Coordinación de casos de uso y orquestación de patrones de software.
    - *Capa de Dominio:* Entidades ricas, reglas ITIL v4 y tres patrones GoF obligatorios (*Factory Method*, *Chain of Responsibility*, *Observer*).
    - *Capa de Persistencia:* Abstracción desacoplada mediante patrón Repository e implementación en memoria/persistencia ligera.
- **Suite de Pruebas:**
    - Pruebas de Caja Negra: Particiones de equivalencia y valores límite sobre las fronteras temporales del SLA (T - 1 s, T, T + 1 s).
    - Pruebas de Caja Blanca: Cobertura exhaustiva de caminos independientes y bifurcaciones en los manejadores de soporte (N1, N2 y N3).

2.4.2 Alcance Excluido (Out-of-Scope)

- **Autenticación Compleja y Roles Distribuidos:** No se implementará SSO, OAuth2/OIDC ni federación con Active Directory institucional (la identidad del operador/usuario se inyecta por cabeceras de contexto).
- **Módulos de Reportería y BI Extensos:** No se construirán dashboards analíticos ni motores de exportación documental (PDF/Excel).
- **Interfaces Web Cosméticas:** Se prescinde de frontends de múltiples vistas; la interacción se valida mediante API REST, OpenAPI y clientes HTTP automatizados.
- **Gestión de Problemas y Cambios ITIL:** El sistema cubre exclusivamente la Gestión de Incidentes; no incluye Change Management ni Problem Management.

2.4.3 Restricciones Técnicas y Metodológicas

- **Restricción R-01 (Trazabilidad 1:1):** Cada clase, método e interfaz implementada en código debe corresponderse estrictamente con el Diagrama de Clases de Diseño (PE1) y el Diagrama de Secuencia (PE2).
- **Restricción R-02 (Estándar Notacional):** Todo artefacto visual debe adherirse a OMG UML 2.5.1 y OMG BPMN 2.0 en código PlantUML compilable.
- **Restricción R-03 (Microservicio Autónomo):** La solución debe ser autosuficiente y ejecutable en entorno local con dependencias declaradas en package.json, sin requerir servicios privativos de nube.

2.4.4 Supuestos Operativos

- **Supuesto S-01:** La jornada laboral para el cómputo de SLAs se modela inicialmente en tiempo continuo o lineal parametrizable dentro de la factoría de tickets.
- **Supuesto S-02:** Los niveles de soporte (N1, N2, N3) cuentan con agentes activos en el pool de atención al momento en que el motor transfiere la incidencia.

3. Análisis (Fase de Inicio)

3.1 Requisitos funcionales y no funcionales

Requisitos Funcionales (RF)

Clasificados con prioridad MoSCoW (*Must have*, *Should have*, *Could have*, *Won't have*) conforme al alcance del prototipo.

Código RF	Descripción	Característica ISO/IEC 25010	Prioridad (MoSCoW)	Caso de Uso Vinculado
RF-01	El sistema debe permitir registrar una incidencia capturando título, descripción, categoría (**Hardware, Software, Redes, Accesos**), impacto y urgencia, generando un identificador correlativo único.	Adecuación Funcional	**Must (Obligatorio)**	CU-01: Registrar Incidencia
RF-02	El sistema debe inicializar	Adecuación Funcional	**Must (Obligatorio)**	CU-01: Registrar Incidencia

	polimórficamente la entidad de ticket según su categoría, calculando la prioridad ITIL y la marca temporal exacta de vencimiento del SLA.			
RF-03	El sistema debe proveer un mecanismo de verificación temporal que evalúe periódicamente o bajo demanda si un ticket ha sobrepasado su fecha límite de resolución.	Adecuación Funcional	**Must (Obligatorio)**	CU-02: Escalar Incidencia
RF-04	Al detectar la expiración de un SLA, el sistema debe disparar un evento de alerta hacia los observadores suscritos (notificador en consola/log y canal de eventos.	Adecuación Funcional	**Must (Obligatorio)**	CU-02: Escalar Incidencia
RF-05	El sistema debe delegar el ticket vencido o de alta complejidad a través de una cadena secuencial de manejadores (N1 → N2 → N3), actualizando su estado a **ESCALADO** y	Adecuación Funcional	**Must (Obligatorio)**	CU-02: Escalar Incidencia

	reasignando el nivel técnico.			
RF-06	El sistema debe permitir consultar un ticket por su ID, exponiendo el nivel de soporte asignado, historial de escalamiento, marcas temporales de SLA y estado actual.	Adecuación Funcional	**Should (Deseable)**	CU-03: Consultar Ticket

Requisitos No Funcionales (RNF) estructurados bajo ISO/IEC 25010:2011

Código RNF	**Característica / Subcaracterística ISO/IEC 25010**	**Descripción Técnica y Restricción**	**Métrica / Criterio de Aceptación Cuantitativo**	**Método de Verificación**
RNF-01	**Rendimiento** / Comportamiento temporal	El tiempo de respuesta de la API REST para el endpoint de registro (**POST /tickets**) debe ser óptimo bajo condiciones normales de carga de la red institucional.	Tiempo de respuesta en percentil 95 (P_{95}) ≤ 200 ms con carga concurrente de 50 peticiones/seg.	Prueba de carga con herramienta automatizada.
RNF-02	**Fiabilidad** / Disponibilidad	El servicio central de gestión y monitoreo de tickets debe operar continuamente durante la jornada laboral de la entidad (08:00 a 18:00 hrs).	Disponibilidad mensual ≥ 99.5% en horario laboral. Tiempo medio de recuperación (MTTR) ≤ 15 min ante fallos no fatales.	Monitoreo de uptime y logs de disponibilidad de servicio.

RNF-03	**Mantenibilidad** / Modularidad	El código debe estructurarse en capas limpias y aplicar patrones GoF para incorporar nuevas categorías o manejadores sin modificar el código cliente (OCP).	Cumplimiento del principio Abierto/Cerrado. Ausencia de referencias circulares en el grafo de módulos.	Análisis estático de dependencias y revisión de código.
RNF-04	**Mantenibilidad** / Testeabilidad (*Testability*)	La lógica de negocio nuclear (cálculo de SLA, factoría y cadena de escalamiento) debe validarse con pruebas unitarias automatizadas.	Cobertura de ramas (*branch coverage*) ≥ 90% en la cadena de escalamiento y factoría. Cobertura de líneas ≥ 85% en dominio.	Reporte automatizado de cobertura con test runner.
RNF-05	**Seguridad** / Integridad	El sistema debe validar estrictamente los payloads de entrada en los endpoints REST, rechazando esquemas malformados o categorías no soportadas.	100% de peticiones con datos inválidos deben responder con código HTTP 400 Bad Request estructurado, sin fugar trazas internas.	Pruebas de caja negra con valores límite y particiones inválidas.

3.2 Modelo de procesos de negocio (BPMN 2.0)

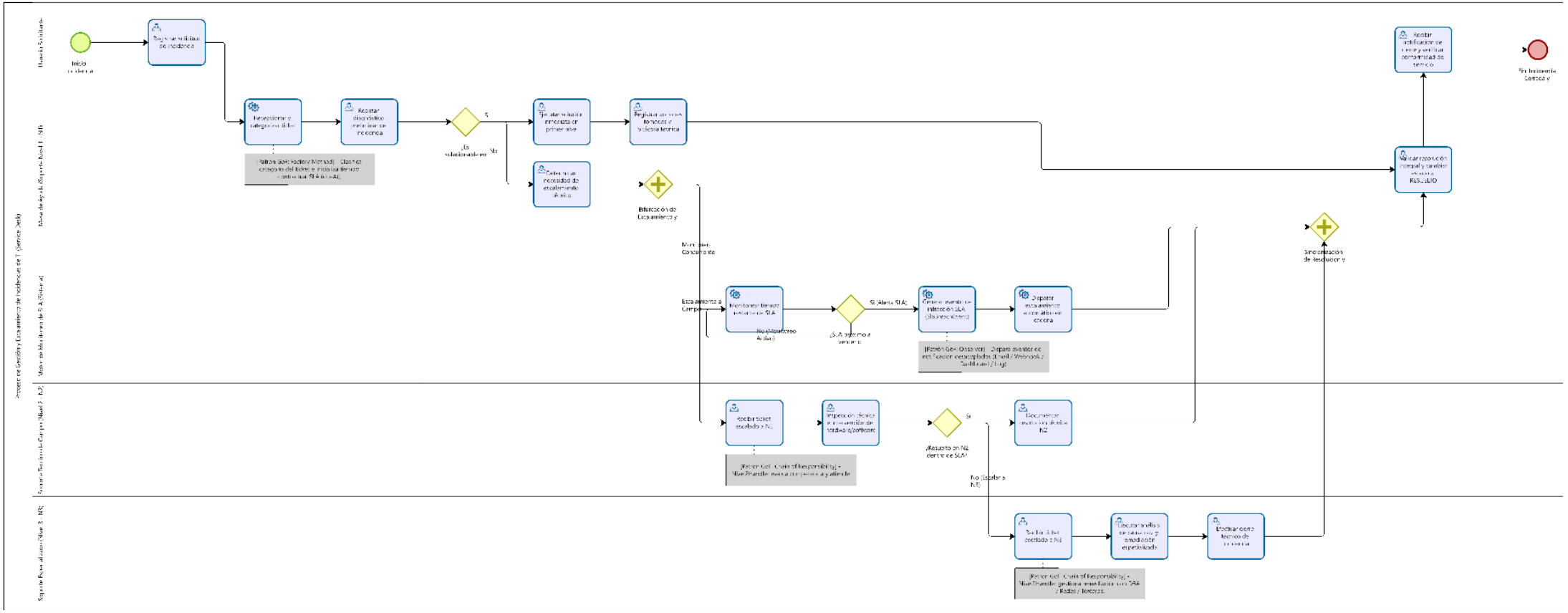
El proceso modelado corresponde a la **Gestión y Escalamiento de Incidencias de TI**, formalizando la interacción entre el Usuario Solicitante, la Mesa de Ayuda (Nivel 1), el Soporte de Campo (Nivel 2), el Soporte Especializado (Nivel 3) y el Motor de Monitoreo Automatizado de SLAs.

Descripción del Flujo Operativo BPMN 2.0

1. **Inicio:** Un usuario de la entidad pública reporta una anomalía tecnológica a través del canal unificado.

2. **Recepción y Triaje (N1):** El operador N1 ingresa la incidencia. El sistema clasifica el tipo (Hardware, Software, Redes, Accesos) y calcula de forma automática la fecha/hora de vencimiento del SLA.
3. **Diagnóstico Preliminar:** Si la incidencia corresponde al primer nivel y puede resolverse en el acto (ej. reseteo de contraseñas), N1 ejecuta la solución y cierra el ticket.
4. **Monitoreo Concurrente de SLA:** En paralelo, el motor de monitoreo evalúa periódicamente el tiempo remanente. Si se detecta un vencimiento o infracción de umbral crítico, se genera un evento de alerta emitido a los observadores suscritos (notificación / log).
5. **Cadena de Escalamiento:** Si el SLA expira o la complejidad técnica supera las competencias de N1, el ticket se transfiere automáticamente a N2. Si N2 no lo resuelve dentro del margen o requiere intervención estructural, se transfiere a N3.
6. **Resolución y Cierre:** El especialista asignado registra la solución técnica y confirma el restablecimiento del servicio al usuario.

Figura 1. **Diagrama BPMN 2.0**



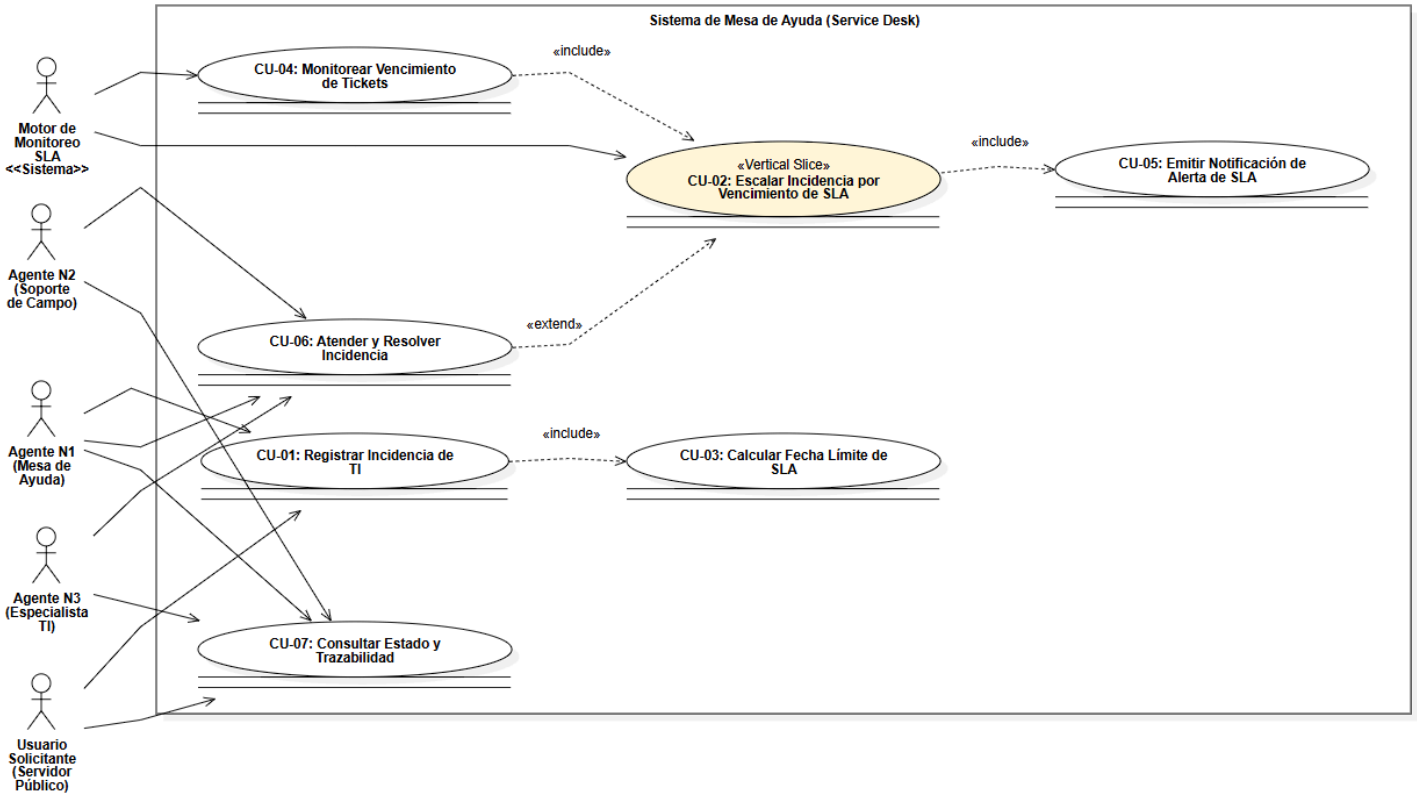
3.3 Modelo de Casos de Uso

El modelo funcional formaliza las interacciones entre los actores del sistema y los servicios provistos, estructurado bajo el estándar OMG UML 2.5.1 mediante relaciones estereotipadas <<include>> y <<extend>>.

Catálogo General de Casos de Uso

- **CU-01:** Registrar Incidencia de TI.
- **CU-02 (Crítico / Vertical Slice):** Escalar Incidencia por Vencimiento de SLA.
- **CU-03:** Calcular Fecha Límite de SLA (<<include>> de CU-01).
- **CU-04:** Monitorear Vencimiento de Tickets (Iniciado por el actor sistema *Motor de Monitoreo SLA*).
- **CU-05:** Emitir Notificación de Alerta de SLA (<<include>> de CU-02).
- **CU-06:** Atender y Resolver Incidencia.
- **CU-07:** Consultar Estado y Trazabilidad.

Figura 2. Modelo de Casos de Uso (UML 2.5.1) - Sistema de Mesa de Ayuda de TI



Especificación del Caso de Uso Crítico (Vertical Slice)

Campo	Detalle Técnico
Identificador	**CU-02**

Campo	Detalle Técnico
Nombre	Registrar Incidencia y Escalarla Automáticamente al Vencer SLA
Actores Primarios	Agente N1 (Mesa de Ayuda) / Servidor Público Solicitante
Actores Secundarios	Motor de Monitoreo de SLA (Sistema), Observadores de Alerta, Agente N2 / N3
Propósito	Capturar una falla de TI, calcular su SLA contractual e iniciar atención; si el tiempo límite expira sin resolución, alertar a los observadores y transferir el ticket al siguiente escalafón de soporte de manera desatendida.
Precondiciones	1. El servicio REST de Service Desk se encuentra activo y disponible. 2. Existen canales observadores de notificación registrados en el sistema. 3. La cadena de escalamiento jerárquica (N1 → N2 → N3) está configurada.
Postcondiciones	1. La incidencia queda persistida con identificador único y marca temporal dueAt calculada. 2. Al verificarse el vencimiento, el estado transiciona a ESCALADO. 3. Se emiten eventos desacoplados hacia los observadores (log y correo). 4. El nivel asignado se incrementa a NIVEL_2 (o NIVEL_3), registrando el motivo en su historial inmutable.

Flujo Básico (Camino Ideal - Vencimiento y Escalamiento)

1. El Agente N1 o Solicitante envía una petición de registro (POST /tickets) suministrando: título, descripción, categoría (Hardware, Software, Redes, Accesos), impacto y urgencia.
2. El sistema valida la integridad de los datos de entrada según las reglas de formato.
3. El sistema resuelve la prioridad ITIL (P1, P2, P3, P4) mediante la matriz de cruce Impacto x Urgencia.
4. El sistema invoca la factoría especializada (TicketFactory) correspondiente a la categoría del ticket.

5. La factoría calcula la fecha y hora exacta de vencimiento contractual: **dueAt = fechaRegistro + SLA_Hours(categoría, prioridad)**
6. El sistema inicializa el ticket en estado ABIERTO, nivel NIVEL_1 y lo persiste en el repositorio.
7. El sistema retorna el ticket creado con código HTTP 201 Created y su carga útil estructurada.
8. El proceso concurrente de monitoreo evalúa periódicamente el estado temporal de los tickets activos.
9. El sistema ejecuta isSlaBreached(currentTime) sobre el ticket y determina que currentTime > dueAt.
10. El sistema instancia un evento SlaBreachEvent y lo publica a través de SlaBreachNotifier (Sujeto del patrón *Observer*).
11. Los observadores suscritos (ConsoleLogObserver, EmailAlertObserver) reciben el evento y despachan las alertas correspondientes de forma desacoplada.
12. El sistema somete el ticket a la cadena de escalamiento (SupportLevelHandler).
13. El manejador actual (L1SupportHandler) evalúa que la incidencia no fue resuelta dentro del SLA y delega la responsabilidad a L2SupportHandler.
14. El manejador L2SupportHandler acepta la delegación, actualiza el estado a ESCALADO, asigna currentLevel = NIVEL_2 e inserta un nuevo EscalationRecord con motivo "SLA_BREACH_TIMEOUT".
15. El sistema persiste la actualización del ticket en el repositorio.
16. El caso de uso finaliza exitosamente garantizando la continuidad de atención.

Flujos Alternos

- **A1: Resolución exitosa en Nivel 1 antes del vencimiento del SLA:**
    - En el paso 8, antes de que el reloj del sistema alcance dueAt, el Agente N1 ejecuta la solución y solicita el cierre (PATCH /tickets/:id/resolve).
    - El sistema actualiza el estado a RESUELTO.
    - El motor de monitoreo omite el ticket en evaluaciones posteriores. El caso de uso finaliza.
- **A2: Escalamiento por complejidad técnica declarada (sin vencimiento previo):**
    - En el paso 8, el analista de N1 diagnostica que la incidencia requiere reemplazo físico de componentes de red troncal.
    - El analista solicita escalamiento manual justificado.
    - La cadena de responsabilidad evalúa la regla de competencia y eleva el ticket a N2 o N3 de forma inmediata, manteniendo el SLA original.

Flujos de Excepción

- **E1: Payload de creación inválido:**
    - En el paso 2, si falta algún atributo obligatorio o la categoría es desconocida, el sistema rechaza la solicitud retornando HTTP 400 Bad Request con el detalle del error. El ticket no se persiste.
- **E2: Falla en canal de notificación de observador:**

- En el paso 11, si el canal de correo presenta timeout o error de red, el observador captura la excepción internamente y registra la traza en el log sin interrumpir ni revertir la transacción de escalamiento del ticket.

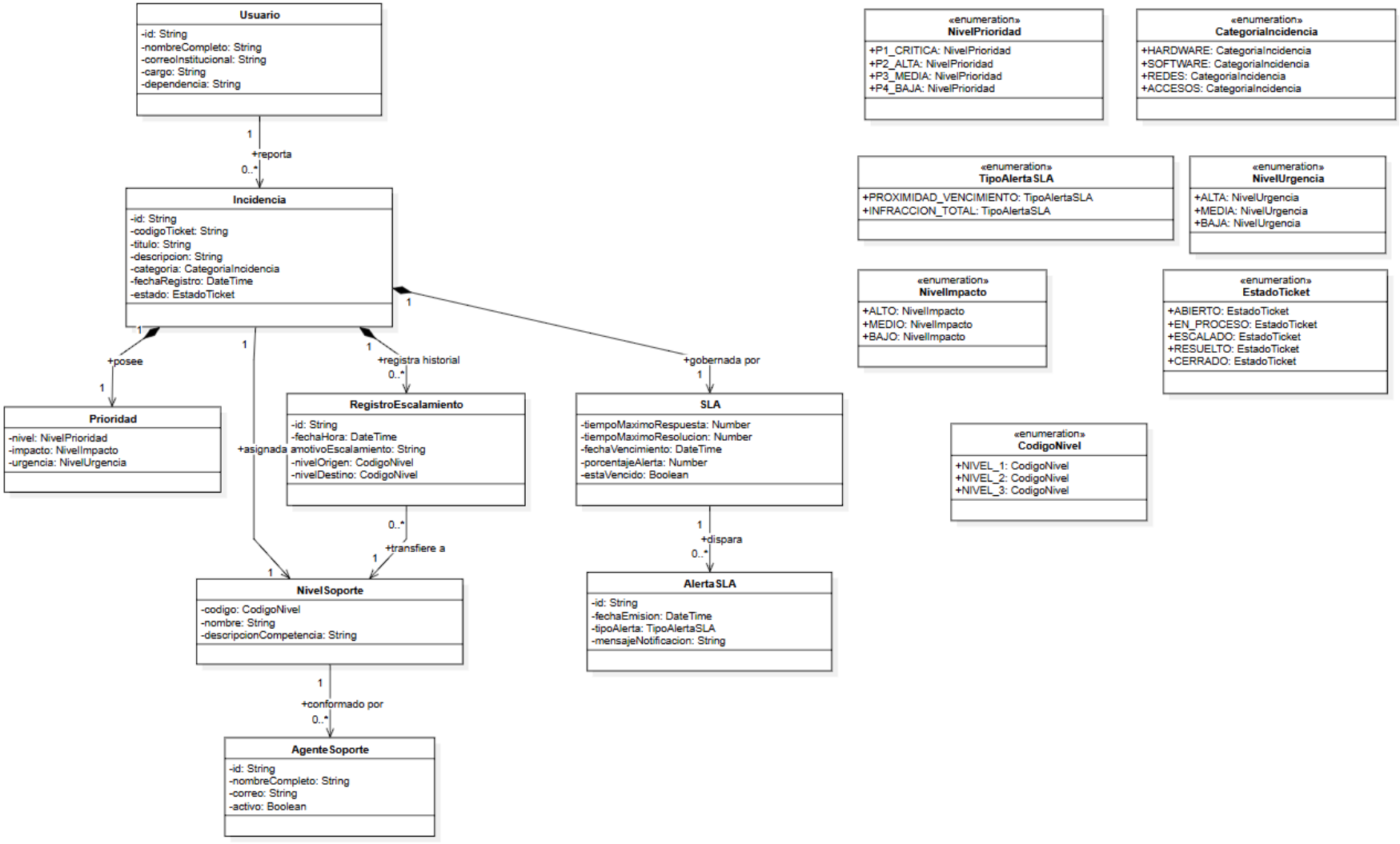
3.4 Modelo de Dominio

El Modelo Conceptual de Dominio representa las abstracciones del negocio y sus interrelaciones estructurales conforme a las directrices de Larman (2004), sin incluir métodos ni detalles de implementación tecnológica.

Diccionario de Clases Conceptuales

- **Usuario (Solicitante):** Servidor público que reporta una interrupción o degradación en sus servicios tecnológicos ante la Mesa de Ayuda.
- **Incidencia (Ticket):** Entidad central que encapsula el requerimiento correctivo. Mantiene su ciclo de vida (ABIERTO, EN_PROCESO, ESCALADO, RESUELTO, CERRADO), su categoría técnica (HARDWARE, SOFTWARE, REDES, ACCESOS) y su historial cronológico.
- **Prioridad:** Entidad resultante del cruce bidimensional ITIL v4 entre **Impacto** y **Urgencia**.
- **SLA (Acuerdo de Nivel de Servicio):** Parámetros contractuales que definen la fecha límite de resolución (dueAt), tiempo límite de primer contacto y umbrales de alerta temprana.
- **NivelSoporte:** Escalafón organizativo de atención (NIVEL_1, NIVEL_2, NIVEL_3) con competencias asignadas.
- **RegistroEscalamiento:** Instancia inmutable que documenta la transferencia entre niveles (marca de tiempo, motivo y actor/mecanismo que autoriza).
- **AlertaSLA:** Evento de notificación emitido ante la proximidad o infracción del tiempo contractual de resolución.

Figura 3. Modelo Conceptual de Dominio - Sistema de Mesa de Ayuda de TI (Service Desk)



4. Diseño (Fase de Elaboración)

4.1 Arquitectura de software - Vista 4+1

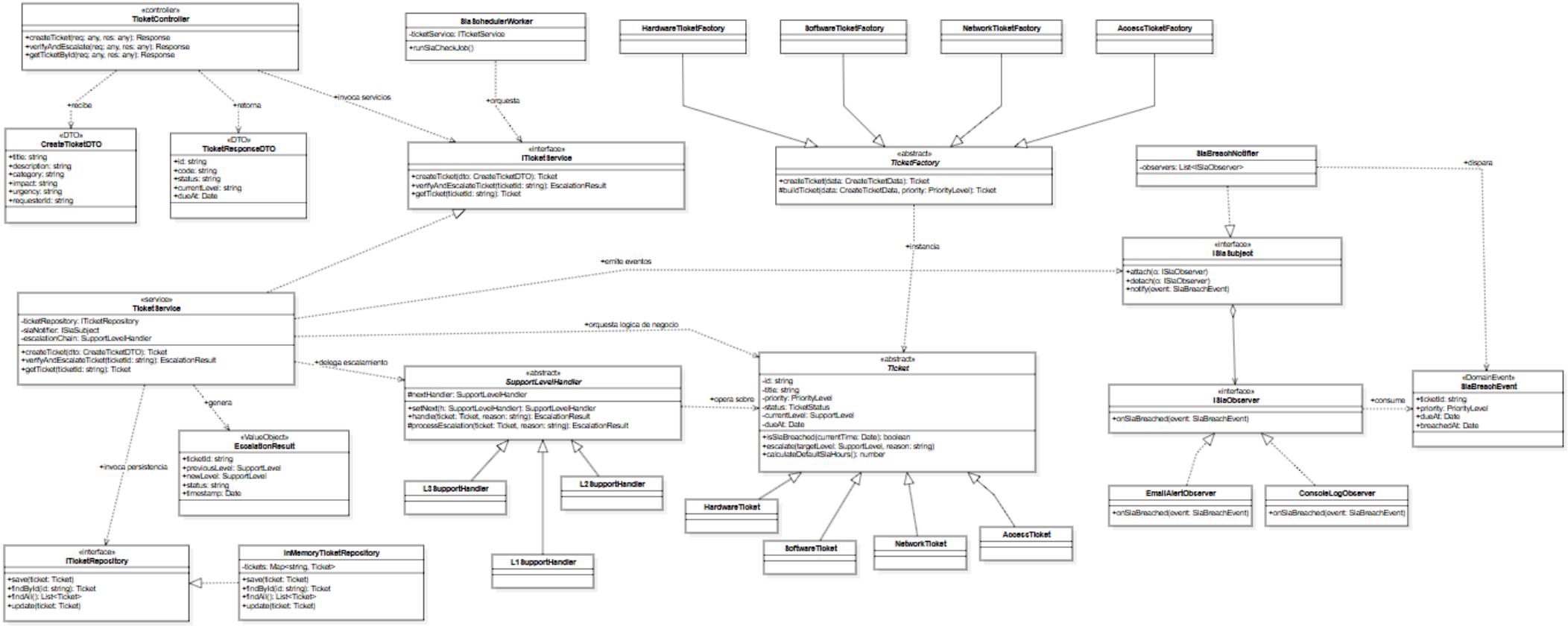
En observancia del estándar ISO/IEC/IEEE 42010:2011 y el modelo de Kruchten (1995), la arquitectura se documenta a través de cuatro vistas fundamentales integradas por el escenario de uso crítico.

4.1.1. Vista Lógica (Logical View)

Descompone el sistema en capas cohesivas y paquetes desacoplados:

- **Presentation:** Controladores REST (TicketController), adaptadores HTTP y objetos de transferencia de datos (CreateTicketDTO).
- **Application:** Servicios de aplicación (TicketService) que coordinan casos de uso sin acoplamiento a infraestructura.
- **Domain:** Núcleo puro del negocio que encapsula entidades (Ticket), factorías polimórficas (TicketFactory), cadena de soporte (SupportLevelHandler), notificador de SLAs (SlaBreachNotifier) e interfaces de persistencia (ITicketRepository).
- **Infrastructure:** Adaptadores tecnológicos (InMemoryTicketRepository, ConsoleLogObserver, EmailAlertObserver, SlaSchedulerWorker).

Figura 4. Vista Lógica de la Arquitectura (Kruchten 4+1) - Service Desk

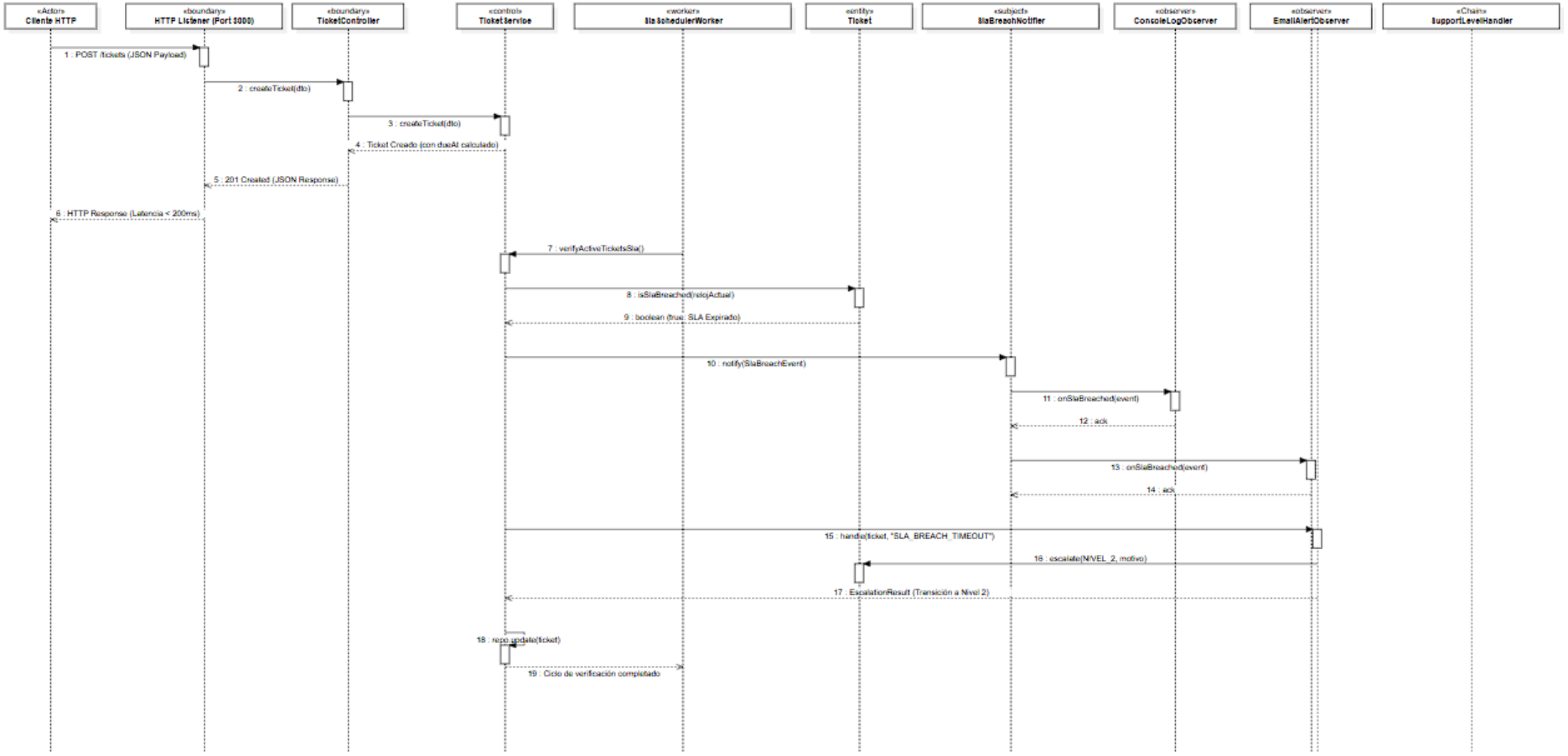


4.1.2. Vista de Procesos (Process View)

Aborda la concurrencia, sincronización de flujos y tiempo de ejecución:

- **Event Loop Principal (Node.js):** Atiende peticiones HTTP de forma síncrona/no bloqueante, asegurando tiempos de respuesta menores a 20 ms para la creación de tickets.
- **Worker Concurrente (SlaSchedulerWorker):** Temporizador desacoplado que evalúa en segundo plano los tickets activos y despacha eventos ante infracciones de SLA sin bloquear las peticiones REST.
- **Despacho Concurrente a Observadores:** Notificación paralela a los observadores registrados, aislando cualquier latencia de I/O de la cadena de escalamiento.

Figura 5. Vista de Procesos (Kruchten 4+1) - Concurrencia y Flujo Asíncrono de SLAs

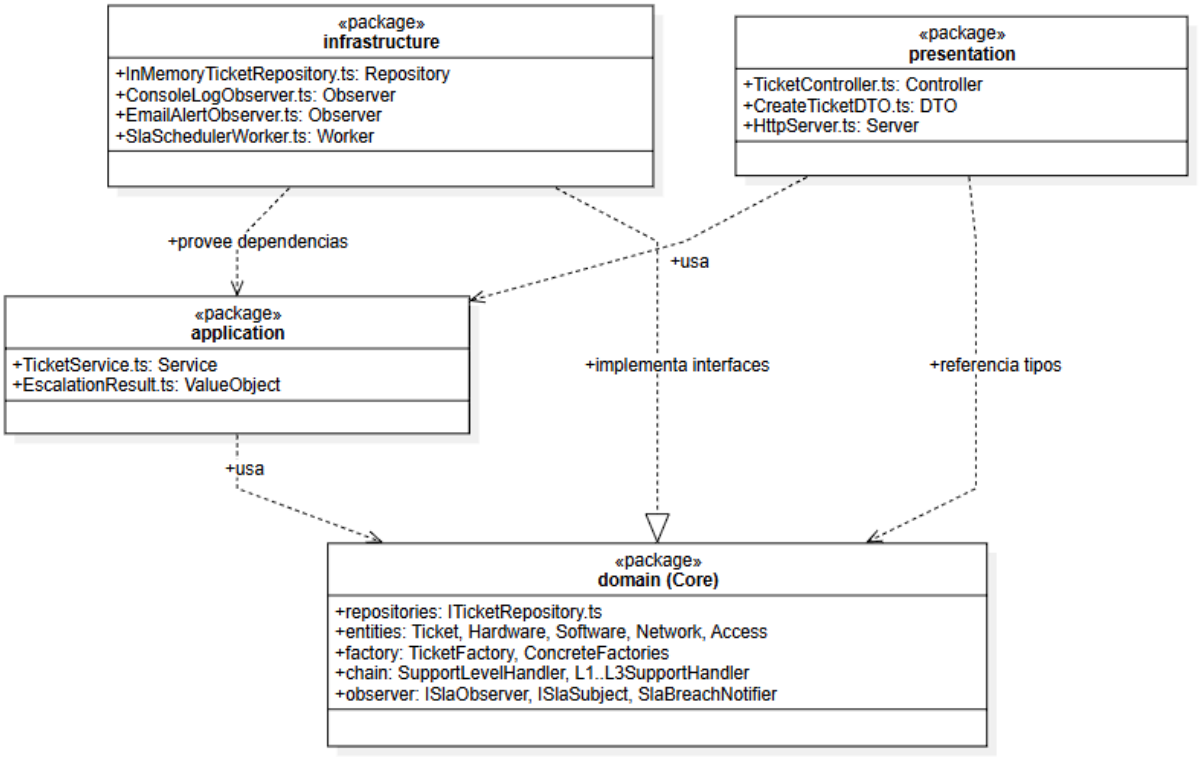


4.1.3. Vista de Desarrollo (Development View)

Define la estructura modular del código bajo principios de Arquitectura Limpia:

- **Regla de Dependencia:** La capa de Dominio no posee dependencias externas. Aplicación depende únicamente de Dominio. Presentación e Infraestructura dependen de las capas internas mediante Inversión de Dependencias (DIP).
- **Organización de Directorios:**
  - src/presentation/: Controladores y definición de servidor HTTP.
  - src/application/: Servicios coordinadores y DTOs de salida.
  - src/domain/: Entidades, factorías GoF, cadena GoF, observadores GoF y contratos de repositorio.
  - src/infrastructure/: Repositorios concretos, observadores de notificación y workers.
  - tests: Pruebas de caja negra y caja blanca.

Figura 6. Vista de Desarrollo (Kruchten 4+1) - Estructura de Paquetes y Módulos



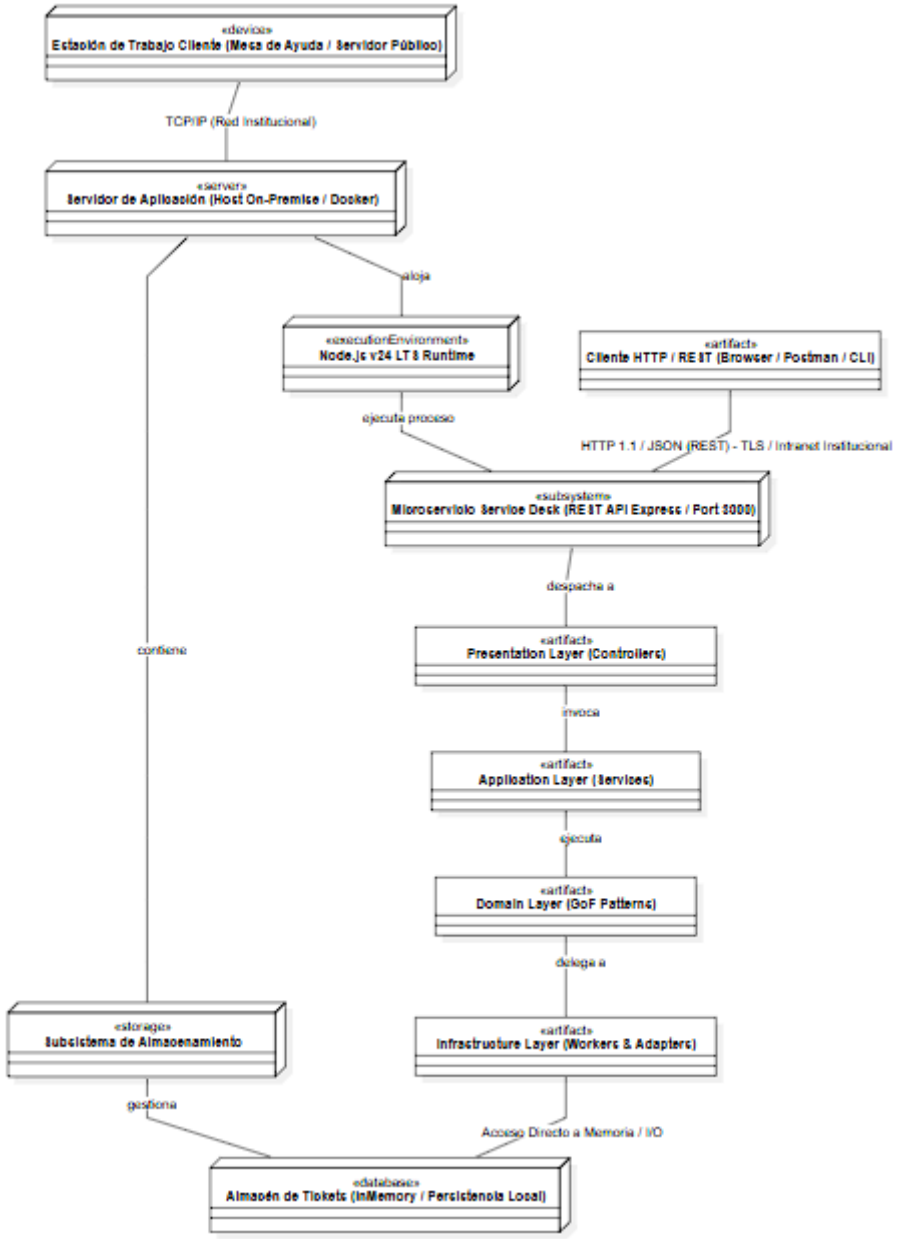
4.1.4. Vista Física / Despliegue (Physical View)

Modela la topología de ejecución del prototipo:

- **Estación Cliente:** Navegador / Postman / Consola emitiendo peticiones HTTP/JSON hacia la API.
- **Servidor de Aplicación (Host / Contenedor):** Proceso Node.js v24 LTS en el puerto TCP 3000 ejecutando la API REST y el worker temporal de SLAs en el mismo espacio de memoria seguro.

- **Subsistema de Persistencia:** Acceso desacoplado a través de ITicketRepository, resuelto en memoria concurrente (InMemoryTicketRepository) para garantizar mínima latencia en el prototipo vertical.

Figura 7. Vista Física - Despliegue (Kruchten 4+1) - Topología de Ejecución



4.1.5. Escenarios (+1)

El escenario central que articula y conecta las cuatro vistas es el **CU-02 (Escalamiento automático por vencimiento de SLA)**:

1. Una petición HTTP entra por la **Vista Física**, es recibida por el Event Loop de la **Vista de Procesos** y canalizada por los controladores de la **Vista de Desarrollo**.
2. Las clases del dominio en la **Vista Lógica** calculan el SLA y persisten el ticket.

3. El worker en segundo plano (**Vista de Procesos**) detecta la expiración temporal e interactúa con el patrón *Observer* y la *Chain of Responsibility* (**Vista Lógica**), propagando el cambio de estado a través de los módulos definidos (**Vista de Desarrollo**).

Justificación de Decisiones Arquitectónicas y Tácticas (SEI / ADD)

Conforme a la metodología de Bass, Clements y Kazman (SEI):

- **Tácticas para Rendimiento (RNF-01: P95 ≤ 200 ms):**
  - *Cálculo estático anticipado:* La factoría calcula la fecha absoluta dueAt al crear el ticket, permitiendo verificaciones de vencimiento en tiempo constante O(1) (now > dueAt).
  - *Persistencia indexada en memoria:* Uso de Map<string, Ticket> con costo de acceso y actualización en tiempo constante O(1).
- **Tácticas para Disponibilidad (RNF-02: Uptime 99.5%):**
  - *Aislamiento de excepciones en observadores:* Cada observador captura fallos externos (ej. caída del servidor SMTP) evitando que se interrumpa la transacción de escalamiento.
  - *Degradación elegante:* El fallo de un canal de notificación no detiene la transferencia de soporte en la cadena de responsabilidad.
- **Tácticas para Mantenibilidad y Testeabilidad (RNF-03 / RNF-04):**
  - *Encapsulamiento de variaciones:* Nuevas categorías de tickets o niveles de soporte se agregan extendiendo clases sin modificar el código cliente (principio Abierto/Cerrado).
  - *Inyección de dependencias:* Los repositorios y manejadores se inyectan en el constructor de los servicios, facilitando el aislamiento para pruebas unitarias.

4.2 Diagrama de clases de diseño

A diferencia del modelo conceptual de dominio, el Diagrama de Clases de Diseño formaliza la implementación orientada a objetos en UML 2.5.1, detallando métodos, tipos de datos, visibilidad y distinguiendo agregaciones de composiciones.

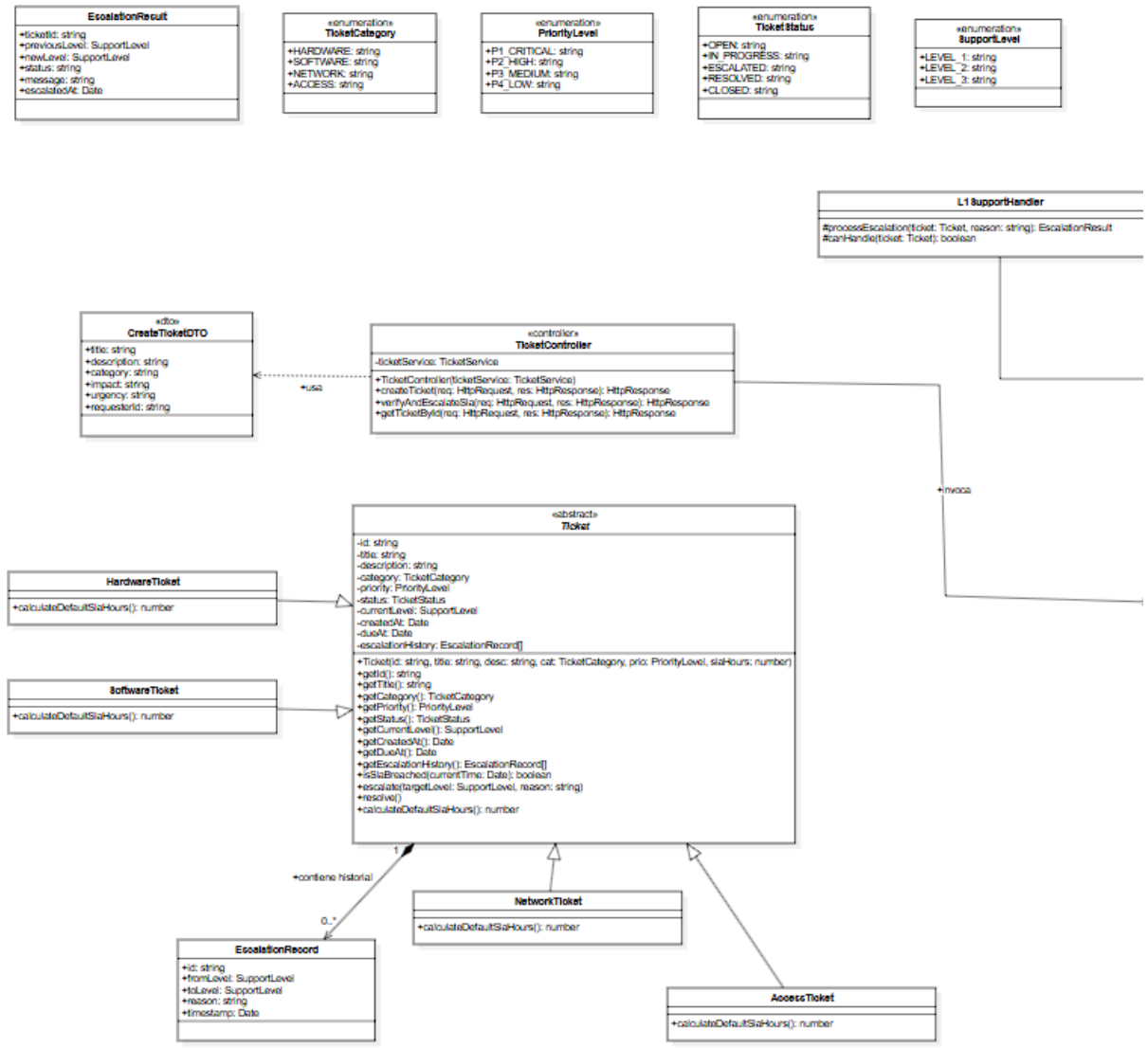
Estructura y Patrones Implementados

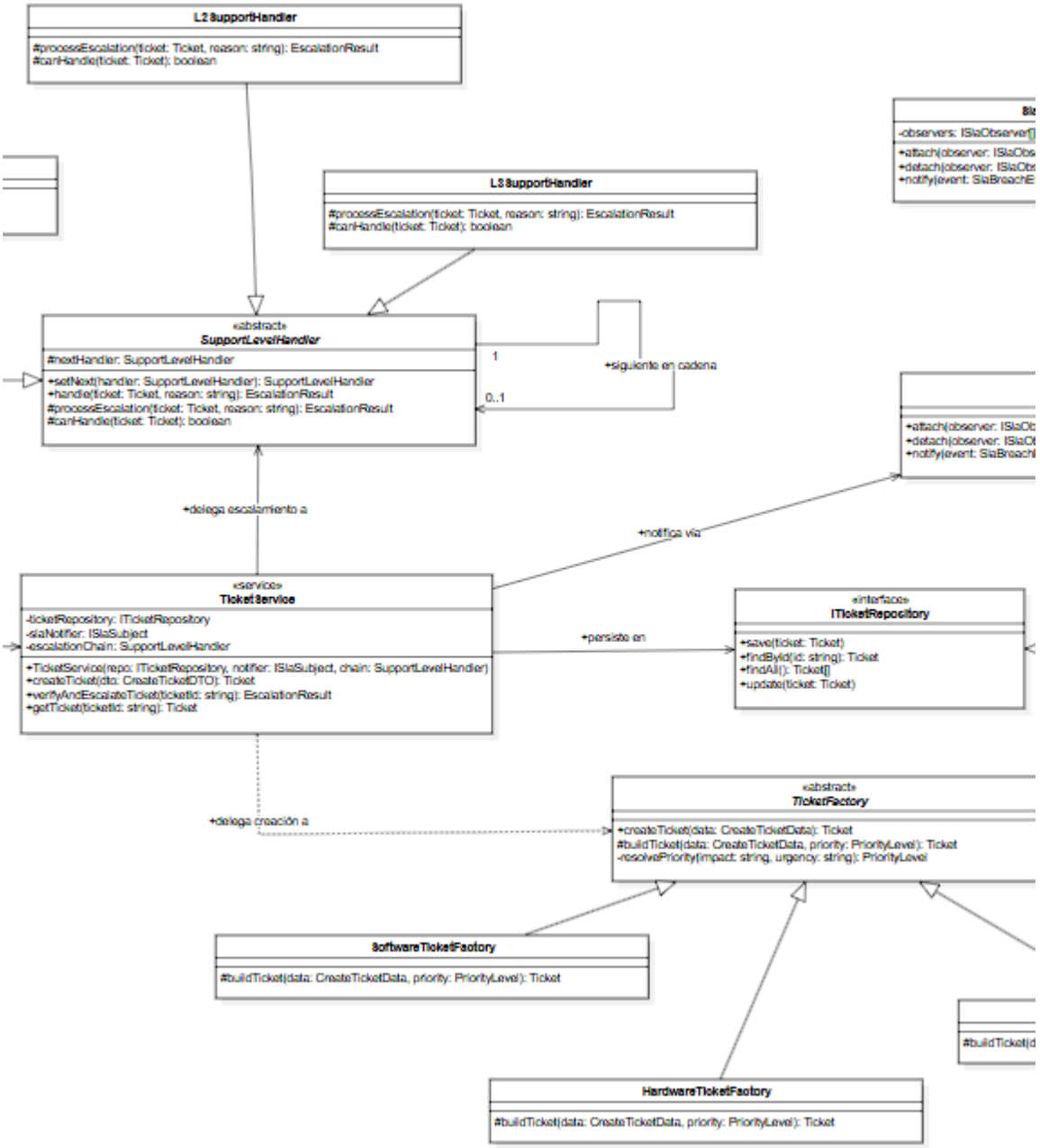
1. **Jerarquía Polimórfica (Factory Method):**
   - Clase abstracta Ticket con atributos privados (id, title, description, status, priority, currentLevel, dueAt, createdAt).
   - Especializaciones: HardwareTicket, SoftwareTicket, NetworkTicket, AccessTicket.
   - Creadores: TicketFactory como creador base y factorías concretas por categoría que encapsulan el cálculo del SLA.
2. **Cadena de Escalamiento (Chain of Responsibility):**
   - Clase base abstracta SupportLevelHandler con referencia nextHandler: SupportLevelHandler y método handle(ticket: Ticket): EscalationResult.

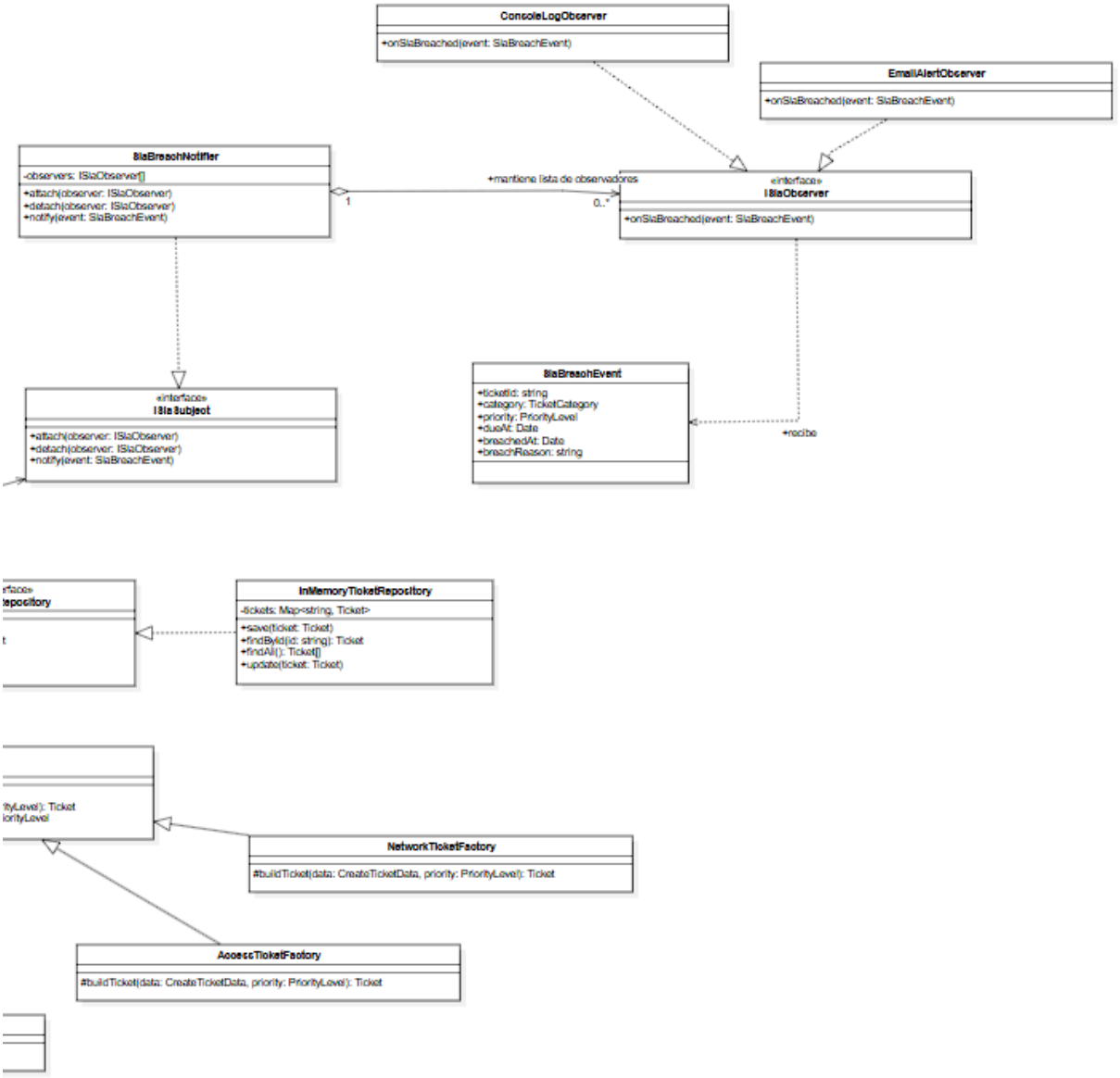
- Eslabones concretos: L1SupportHandler, L2SupportHandler y L3SupportHandler.

3. **Mecanismo de Alertas (Observer):**
   - Interfaz ISlaSubject implementada por SlaBreachNotifier.
   - Interfaz ISlaObserver con método update(event: SlaBreachEvent): void, implementada por ConsoleLogObserver y EmailAlertObserver.
4. **Capa de Persistencia:**
   - Interfaz ITicketRepository (save, findById, findAll, findActiveTickets).
   - Implementación concreta InMemoryTicketRepository.

Figura 8. Diagrama de Clases de Diseño (UML 2.5.1) - Microservicio Mesa de Ayuda TI




L2SupportHandler
#processEscalation(ticket: Ticket, reason: string): EscalationResult
#canHandle(ticket: Ticket): boolean
L3SupportHandler
#processEscalation(ticket: Ticket, reason: string): EscalationResult
#canHandle(ticket: Ticket): boolean
«abstract»
SupportLevelHandler
#nextHandler: SupportLevelHandler
+setNext(handler: SupportLevelHandler): SupportLevelHandler
+handle(ticket: Ticket, reason: string): EscalationResult
#processEscalation(ticket: Ticket, reason: string): EscalationResult
#canHandle(ticket: Ticket): boolean
1
0..1
+siguiente en cadena
+delega escalamiento a
+notifica vía
«service»
TicketService
-ticketRepository: ITicketRepository
-slaNotifier: ISlaSubject
-escalationChain: SupportLevelHandler
+TicketService(repo: ITicketRepository, notifier: ISlaSubject, chain: SupportLevelHandler)
+createTicket(dto: CreateTicketDTO): Ticket
+verifyAndEscalateTicket(ticketId: string): EscalationResult
+getTicket(ticketId: string): Ticket
+persiste en
«interface»
ITicketRepository
+save(ticket: Ticket)
+findById(id: string): Ticket
+findAll(): Ticket[]
+update(ticket: Ticket)
+delega creación a
«abstract»
TicketFactory
+createTicket(data: CreateTicketData): Ticket
#buildTicket(data: CreateTicketData, priority: PriorityLevel): Ticket
-resolvePriority(impact: string, urgency: string): PriorityLevel
SoftwareTicketFactory
#buildTicket(data: CreateTicketData, priority: PriorityLevel): Ticket
HardwareTicketFactory
#buildTicket(data: CreateTicketData, priority: PriorityLevel): Ticket

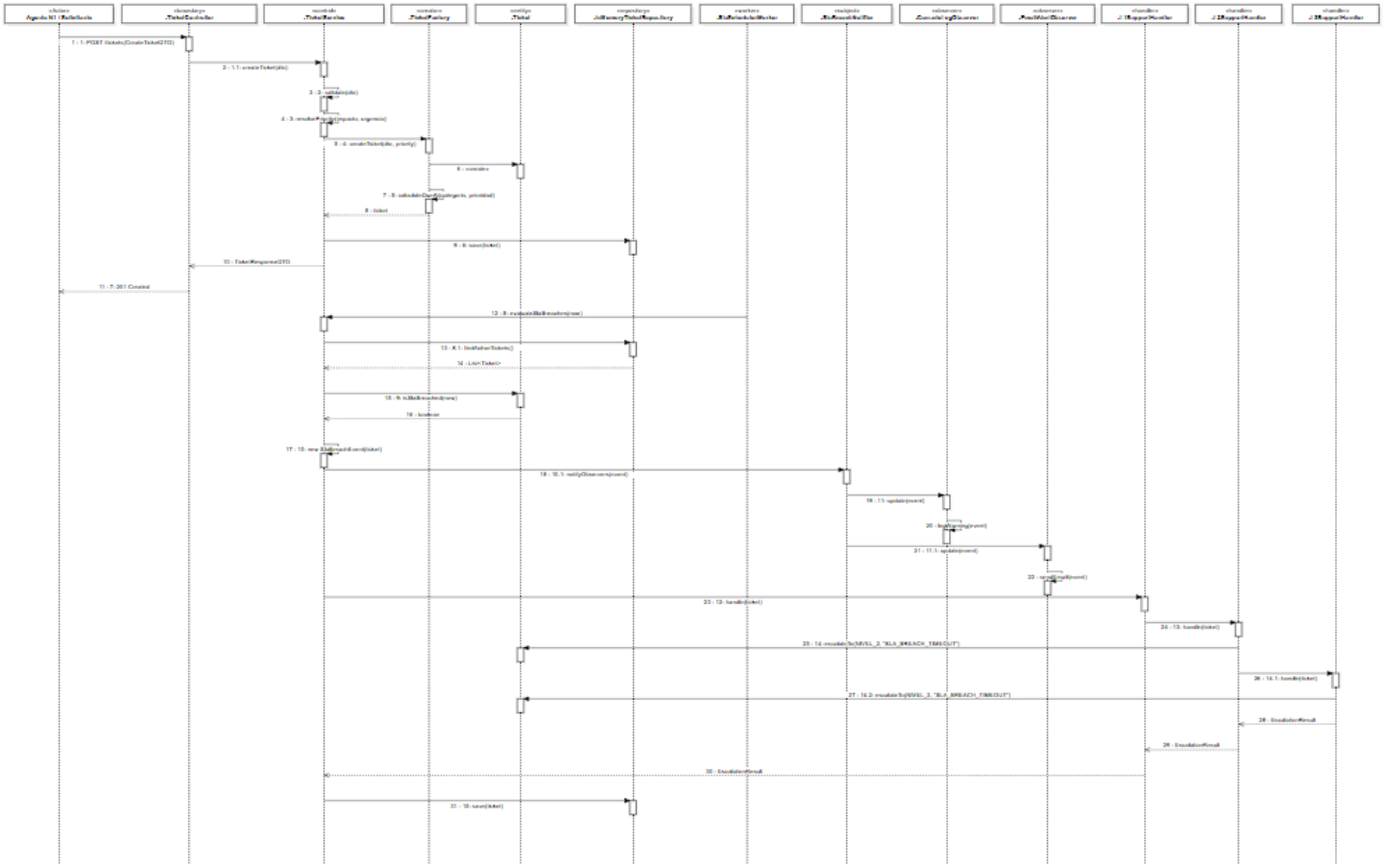


4.3 Diagrama de secuencia

Conforme a la especificación OMG UML 2.5.1 y el estándar ISO/IEC/IEEE 42010:2011, el modelado dinámico formaliza la colaboración temporal y el intercambio de mensajes entre los objetos del sistema durante la ejecución del Caso de Uso Crítico (CU-02: Registrar incidencia y escalarla automáticamente al vencer SLA).

La interacción se estructura aplicando la separación de responsabilidades en objetos de Frontera (<<boundary>>), Control (<<control>>) y Entidad (<<entity>>), articulados armónicamente con los tres patrones de diseño GoF obligatorios (Factory Method, Observer y Chain of Responsibility).

Figura 9. Diagrama de Secuencia - CUS 02 (UML 2.5.1) - Microservicio Mesa de Ayuda TI



4.3.1 Catálogo de Líneas de Vida Participantes

Clasificador / Línea de Vida	Estereotipo / Rol	Responsabilidad en la Interacción Dinámica
Agente N1 / Solicitante	Actor	Emite la solicitud HTTP de registro y recibe el acuse de creación del ticket.
:TicketController	<<boundary>>	Punto de entrada REST; des-serializa y valida el payload HTTP (CreateTicketDTO) y serializa la respuesta (201 Created).
:TicketService	<<control>>	Orquestador de la lógica de aplicación; coordina la factoría, la persistencia, el notificador y la cadena de escalamiento.

:TicketFactory	<<creator>> (GoF)	Factoría especializada; resuelve la prioridad ITIL y calcula analíticamente la fecha límite contractual (dueAt).
:Ticket	<<entity>>	Instancia de dominio; encapsula el estado (ABIERTO, ESCALADO), evalúa isSlaBreached() y ejecuta el cambio de nivel.
:InMemoryTicketRepository	<<repository>>	Puerto de infraestructura; garantiza la persistencia y recuperación eficiente O(1) de las entidades en memoria.
:SlaSchedulerWorker	<<worker>>	Proceso en segundo plano desacoplado; dispara la verificación periódica de vencimiento sobre los tickets activos.
:SlaBreachNotifier	<<subject>> (GoF)	Sujeto emisor de eventos; gestiona la lista de suscriptores y despacha concurrentemente los eventos de infracción.
:ConsoleLogObserver	<<observer>> (GoF)	Canal observador concreto; captura el evento y escribe la traza de advertencia en los logs de auditoría.
:EmailAlertObserver	<<observer>> (GoF)	Canal observador concreto; despacha alertas telemáticas al personal de soporte y jefaturas.
:L1SupportHandler	<<handler>> (GoF)	Eslabón inicial de la cadena; detecta la expiración en primer nivel y delega al siguiente manejador.
:L2SupportHandler	<<handler>> (GoF)	Eslabón intermedio; asume la responsabilidad técnica, eleva el nivel a NIVEL_2 o delega a N3 si persiste el fallo.
:L3SupportHandler	<<handler>> (GoF)	Eslabón terminal; gestiona

		incidentes de máxima criticidad con especialistas corporativos y proveedores.

4.3.2 Justificación de Fragmentos Combinados (UML 2.5.1)

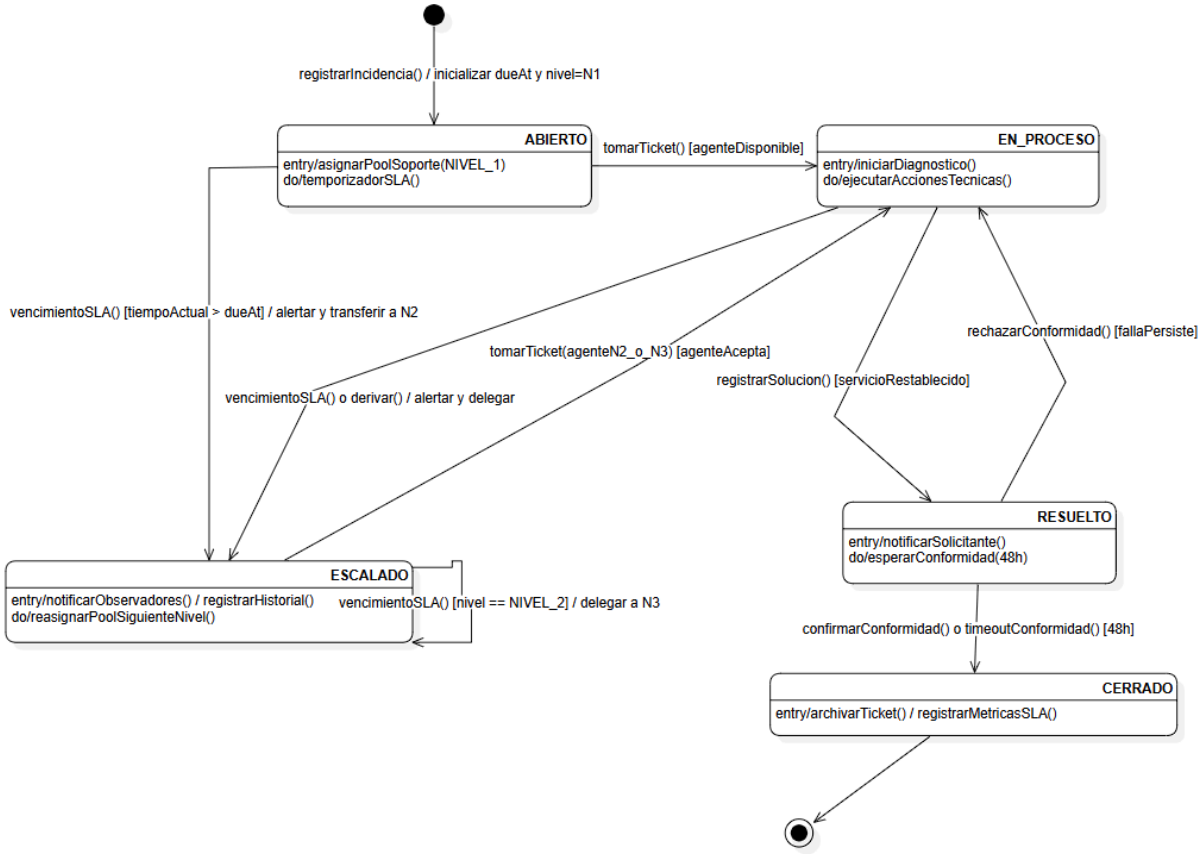
El diagrama implementa rigurosamente los siguientes operadores de interacción combinada:

- **loop (Ciclo de Evaluación Temporal):** Modela la inspección iterativa que ejecuta el worker sobre la colección de tickets abiertos.
- **alt (Bifurcación de Vencimiento y Competencia de Nivel):** Evalúa la condición de guarda [now > dueAt]. Si es verdadera, dispara la alerta y el escalamiento; en caso contrario, mantiene el monitoreo pasivo. Asimismo, discrimina la resolución entre el Nivel 2 y el Nivel 3.
- **par (Notificación Paralela a Observadores):** Permite el despacho concurrente de alertas hacia múltiples canales independientes (Consola y Correo) sin inducir bloqueos ni retardos acumulativos en el hilo de aplicación.
- **opt (Acciones de Contingencia):** Maneja la captura de excepciones y generación de trazas de auditoría ante eventuales indisponibilidades temporales de los canales telemáticos externos.

4.4 Diagrama de estados

Tomando como referencia el estándar OMG UML 2.5.1 y las buenas prácticas de ITIL v4, especialmente las relacionadas con la gestión de incidentes y los acuerdos de nivel de servicio (SLA), se representa el comportamiento y las diferentes etapas por las que pasa una incidencia de TI desde su registro hasta su cierre.

Figura 10. Diagrama de Estados (UML 2.5.1) - Microservicio Mesa de Ayuda TI



A diferencia de un diagrama de procesos, que muestra la secuencia de actividades, el diagrama de estados permite identificar las distintas situaciones por las que atraviesa un ticket durante su ciclo de vida. También permite representar las acciones que se ejecutan al ingresar, permanecer o salir de un estado, así como las condiciones y los eventos que provocan los cambios. Estas transiciones pueden producirse por acciones de los usuarios o del personal técnico, así como por las tareas automáticas del sistema de monitoreo.

4.4.1. Descripción de los estados y sus actividades internas

1. **ABIERTO**

- **Descripción:** Es el estado inicial de la incidencia una vez que se ha registrado correctamente en el sistema. En esta etapa, el ticket ya cuenta con un identificador único, ha sido clasificado según sus características y tiene una fecha límite de atención (dueAt) establecida de acuerdo con el SLA correspondiente.
- **Actividades internas:**
    - entry / asignarPoolSoporte(NIVEL_1): Al ingresar a este estado, el sistema asigna la incidencia a la bandeja de atención del equipo de soporte de Nivel 1.
    - do / temporizadorSLA(): Mientras el ticket permanece abierto, el sistema supervisa el tiempo transcurrido para verificar que la atención se realice dentro del plazo establecido.

2. **EN_PROCESO**

- **Descripción:** Este estado indica que la incidencia ya está siendo atendida por un analista técnico. El responsable puede pertenecer al Nivel 1, Nivel 2 o Nivel 3, dependiendo de las características del problema y de la complejidad de su solución.
- **Actividades internas:**
    - entry / iniciarDiagnostico(): Al comenzar la atención, se registra la marca de tiempo del primer contacto con la incidencia, lo que permite medir el tiempo de respuesta y evaluar el cumplimiento del SLA.
    - do / ejecutarAccionesTecnicas(): El analista realiza las acciones necesarias para identificar y solucionar el problema, como corregir configuraciones, reinstalar software, restablecer perfiles o solucionar fallas de conectividad.

3. ESCALADO (Estado Crítico del *Vertical Slice*)

- **Descripción:** Este estado se alcanza cuando la incidencia necesita ser atendida por un nivel de soporte superior. Esto puede ocurrir porque el tiempo establecido en el SLA ha vencido o porque el problema presenta una complejidad que supera las capacidades del equipo responsable en ese momento.
- **Actividades internas:**
    - entry / notificarObservadores(): Al ingresar a este estado, el sistema genera notificaciones mediante el componente SlaBreachNotifier, siguiendo el patrón *Observer*. Estas alertas pueden registrarse en los logs de auditoría y enviarse por correo electrónico o mediante un webhook.
    - entry / registrarHistorial(): Se registra la escalación en EscalationRecord, incluyendo la fecha y hora, el nivel de soporte de origen, el nivel de destino y el motivo del cambio. Entre las causas contempladas se encuentran SLA_BREACH_TIMEOUT y TECHNICAL_COMPLEXITY.
    - do / reasignarPoolSiguienteNivel(): El sistema transfiere la incidencia al siguiente nivel de soporte según corresponda, por ejemplo, del Nivel 1 al Nivel 2 o del Nivel 2 al Nivel 3.

4. RESUELTO

- **Descripción:** La incidencia llega a este estado cuando el personal técnico ha aplicado las medidas correctivas necesarias y ha conseguido restablecer el funcionamiento del servicio afectado. Sin embargo, todavía queda pendiente confirmar que la solución funcione correctamente para el usuario solicitante.
- **Actividades internas:**
    - entry / notificarSolicitante(): El sistema envía una notificación automática al servidor público que registró la incidencia, informándole sobre la solución aplicada.
    - do / esperarConformidad(48h): Se establece un periodo de 48 horas hábiles para que el usuario compruebe si el servicio funciona con normalidad y confirme que el problema ha sido solucionado.

5. CERRADO

- **Descripción:** Es el estado final del ciclo de vida operativo del ticket. Se alcanza cuando la incidencia se considera concluida y se han cumplido las condiciones establecidas para su cierre. A partir de ese momento, el registro queda protegido frente a modificaciones ordinarias.
- **Actividades internas:**
    - entry / archivarTicket(): El sistema archiva el ticket y restringe su edición, permitiendo únicamente la consulta de la información registrada.
    - entry / registrarMetricasSLA(): Se actualizan los indicadores de gestión de la OGTI, incluyendo el tiempo medio de resolución (MTTR), el porcentaje de cumplimiento del SLA de resolución (SLA-T) y el rendimiento de los distintos niveles de soporte.

4.4.2. Matriz detallada de transiciones, disparadores y guardas

La siguiente tabla describe las transiciones entre los estados del ticket, especificando los eventos que las activan, las condiciones que deben cumplirse y las acciones que ejecuta el sistema como consecuencia del cambio.

Para representar estas transiciones se utiliza la sintaxis de UML: Evento(Parámetros) [Condición de Guarda] / Acción de Efecto. De esta manera, se puede identificar qué provoca cada cambio de estado, qué requisitos deben cumplirse para que ocurra y qué operaciones se realizan durante la transición.

Estado Origen	**Estado Destino**	**Evento Disparador (*Trigger*)**	**Condición de Guarda (*Guard*)**	**Acción de Efecto (*Effect*)**	**Justificación ITIL / Técnica**
● (Inicio)	ABIERTO	registrarIncidencia(datos)	N/A	/ inicializar dueAt y nivel = NIVEL_1	El solicitante o N1 registra la falla. La factoría computa la prioridad ITIL (P1 a P4) y la hora de vencimiento dueAt.
ABIERTO	EN_PROCESO	tomarTicket()	[agenteDisponible]	N/A	Un operador de Nivel 1 asume la atención del requerimiento.
ABIERTO	ESCALADO	vencimientoSLA()	[tiempoActual > dueAt]	/ alertar y transferir a N2	**Escalamiento reactivo en bandeja:** El ticket expiró sin siquiera haber sido tomado en N1. Se eleva forzosamente a Nivel 2.
EN_PROCESO	ESCALADO	vencimientoSLA()	[tiempoActual > dueAt]	/ alertar y delegar	**Escalamiento automático (*Vertical Slice*):** El agente N1 no resolvió la falla dentro del SLA-T. El temporizador transfiere el ticket a N2.

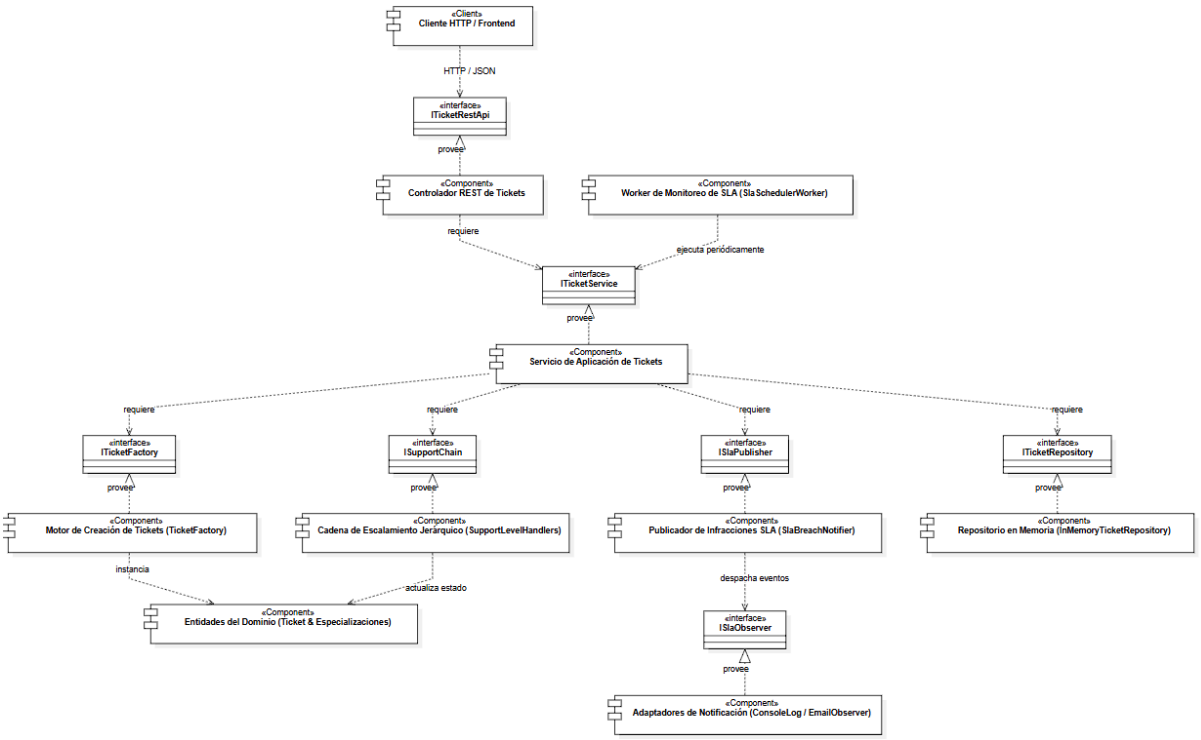
EN_PROCESO	ESCALADO	derivarPorComplejidad()	[superaCompetenciaN1]	/ alertar y delegar	**Escalamiento técnico funcional:** Diagnóstico temprano detecta falla física o de infraestructura troncal fuera del alcance de N1.
ESCALADO	EN_PROCESO	tomarTicket(agenteN2_o_N3)	[agenteAcepta]	N/A	Un técnico del nivel superior toma el ticket escalado para proseguir la atención especializada.
ESCALADO	ESCALADO	vencimientoSLA()	[tiempoActual > dueAt && nivel == NIVEL_2]	/ delegar a N3	**Escalamiento recursivo / Re-escalamiento:** Si el ticket ya en Nivel 2 vuelve a infringir el SLA máximo, la cadena de responsabilidad lo eleva a Nivel 3.
EN_PROCESO	RESUELTO	registrarSolucion()	[servicioRestablecido]	N/A	El especialista ingresa la causa raíz y la solución implementada.
RESUELTO	EN_PROCESO	rechazarConformidad()	[fallaPersiste]	N/A	**Flujo de Excepción (Reapertura):** El usuario reporta dentro de las 48h que el servicio continúa degradado. Regresa a diagnóstico técnico.
RESUELTO	CERRADO	confirmarConformidad()	N/A	N/A	El funcionario emite conformidad explícita con el servicio restablecido.
RESUELTO	CERRADO	timeoutConformidad()	[plazoExpirado (48h)]	N/A	Cierre administrativo automático por inacción del usuario transcurrido el tiempo límite normativo.
CERRADO	⊙ (Fin)	Transición inmediata	N/A	N/A	Conclusión formal del ciclo de vida de la entidad en el microservicio.

4.5 Diagrama de Componentes

Conforme a los lineamientos de la arquitectura de software descrita bajo ISO/IEC/IEEE 42010:2011 y la notación OMG UML 2.5.1, la estructura modular del sistema se formaliza mediante un Diagrama de Componentes. Este artefacto modela las piezas autónomas y reutilizables de software que integran el microservicio, especificando contractualmente sus interfaces provistas (provided interfaces o lollipops -()) y sus interfaces requeridas (required interfaces o sockets ..()).

Esta descomposición modular garantiza una estricta aplicación del Principio de Inversión de Dependencias (DIP) de SOLID y una arquitectura basada en Puertos y Adaptadores (Hexagonal / Clean Architecture), aislando las reglas de negocio del dominio frente a los detalles de presentación, almacenamiento y servicios telemáticos externos.

Figura 11. Diagrama de Componentes (UML 2.5.1) - Service Desk



4.5.1 Catálogo de Componentes e Interfaces del Sistema

Componente	Capa Arquitectónica	Interfaz Provista (Lollipop)	Interfaz Requerida (Socket)	Rol y Responsabilidad Técnica
Cliente HTTP / Frontend	Consumo Externo	-	ITicketRestApi	Aplicación cliente (Postman / SPA / Consola) que consume la API mediante peticiones HTTP/JSON sobre TLS.
Controlador REST de Tickets	Presentación	ITicketRestApi	ITicketService	Adaptador primario de entrada; recibe solicitudes HTTP, valida esquemas DTO y traduce las respuestas del sistema.
Servicio de Aplicación de	Aplicación	ITicketService	ITicketFactory ISupportChain	Núcleo coordinador del microservicio; orquesta

Tickets			ISlaPublisher ITicketRepository	los casos de uso sin contaminarse con detalles de infraestructura ni frameworks web.
Motor de Creación (TicketFactory)	Dominio (GoF)	ITicketFactory	-	Encapsula la lógica de inicialización polimórfica y el cálculo analítico del SLA según la categoría de la incidencia. Instancia las entidades.
Cadena de Escalamiento (Handlers)	Dominio (GoF)	ISupportChain	-	Implementa la jerarquía de niveles de soporte (N1→N2→N3) y evalúa las reglas de transferencia técnica de los tickets.
Publicador de Alertas (SlaBreachNotifier)	Dominio (GoF)	ISlaPublisher	ISlaObserver	Implementa el patrón Observer; administra el registro de suscriptores y desacopla la emisión del evento frente a sus destinatarios.
Entidades del Dominio (Ticket & Types)	Dominio	-	-	Contiene los modelos ricos (Ticket, HardwareTicket, etc.) y objetos de valor que encapsulan el estado y las reglas de negocio ITIL.
Repositorio en Memoria	Infraestructura	ITicketRepository	-	Adaptador de persistencia secundario; provee almacenamiento concurrente de alta velocidad en memoria (Map<string, Ticket>).
Adaptadores de Notificación	Infraestructura	ISlaObserver	-	Adaptadores de salida secundarios (ConsoleLogObserver, EmailAlertObserver) encargados de materializar las alertas hacia el entorno.
Worker de Monitoreo de SLA	Infraestructura	-	ITicketService	Componente activo en segundo plano (timer/cron) que invoca

				periódicamente la verificación de vencimiento sobre la capa de aplicación.

4.5.2 Mecanismo de Acoplamiento y Flujo de Interconexión

La interconexión de componentes se fundamenta en los siguientes principios de ingeniería de software:

- **Aislamiento Absoluto del Dominio:**

Los componentes de dominio (TicketFactory, SupportLevelHandlers, SlaBreachNotifier y DomainEntities) operan en un nivel de abstracción puro. No poseen dependencias hacia librerías de terceros, bases de datos ni protocolos de transporte. Proveen interfaces explícitas (ITicketFactory, ISupportChain, ISlaPublisher) que la capa de aplicación consume directamente.

- **Coordinación Mediante Puertos de Aplicación:**

El componente Servicio de Aplicación de Tickets actúa como mediador. Provee ITicketService hacia los adaptadores de entrada (Controlador REST y Worker de Monitoreo) y consume las interfaces requeridas del dominio y de infraestructura (ITicketRepository).

- **Sustituibilidad y Bajo Acoplamiento (Lollipop & Socket):**

Dado que InMemoryTicketRepository se acopla a través del contrato ITicketRepository, la infraestructura puede migrarse de un repositorio en memoria hacia un motor relacional (ej. PostgreSQL / TypeORM / Prisma) sin modificar una sola línea de código en la capa de aplicación ni en el dominio. De igual forma, pueden incorporarse nuevos observadores telemáticos (Slack, SMS, Webhooks) implementando la interfaz ISlaObserver.

4.5.3 Topología de Despliegue Físico Integrada

En concordancia con la Vista Física (Sección 4.1.4), todos los componentes de software del microservicio se compilan y empaquetan en un único artefacto ejecutable bajo el entorno de ejecución Node.js v24 LTS.

- **Frontera de Red:** El componente Controlador REST de Tickets expone el puerto TCP 3000 hacia la Intranet institucional, recibiendo tráfico estructurado en formato JSON UTF-8.
- **Espacio de Ejecución de Memoria:** Tanto la capa de aplicación, el dominio con sus patrones GoF, el worker de monitoreo de SLAs y el repositorio concurrente coexisten en el mismo proceso de servidor V8, lo que elimina retardos de serialización entre procesos y garantiza tiempos de respuesta de registro y verificación dentro de la cota $P_{95} \leq 200\text{ms}$ (RNF-01).